Any of these operations, in fact, would require a linear number of operations, both in worst cases and in the average case[3].

For this use case, it could probably work. But if our list had millions or billions of elements, then we would most likely be in trouble.

2.3.1 From sorted lists to priority queues

Luckily for us, there is a better solution.

This is the perfect use case for one of the core data structures.

A priority queue will keep a partial ordering of the elements, with the guarantee that the next element returned from the queue will hold the highest priority.

By giving up the requirement of a total ordering (which we wouldn’t need anyway, in this case, as we only consume tasks one by one), we gain in performance: each of the operations on the queue can now require only logarithmic time.

As a side note, this reminds us how important it is to get our requirements right before implementing any solution, and make sure we don’t overcomplicate our work and requirements: keeping a list of elements sorted when all we need is a partial ordering wastes resources, and complicates our code, making it harder to maintain and scale.